PROJECT PROPOSAL

Food Delivery Platform

A cloud-ready, event-driven microservices platform for online food ordering, restaurant operations and last-mile delivery.

Java 21 · Spring Boot 4 | 16 microservices | Apache Kafka | React 19

MySQL · Redis · Elasticsearch | Docker | AI assistant

Architecture

An API gateway, 13 domain services and 2 AI services, each with its own data, integrated through Kafka events.

Security and operations

Role-based access with JWT, resilience patterns, metrics, logs and traces, automated quality gates.

Delivery

18 reviewed phases, five environments, containerised services and a CI/CD path to a managed cloud.

DOCUMENT

Project Proposal: architecture, technology, security and deployment

VERSION

1.0 · 14 October 2026

INTENDED READERS

Client stakeholders, solution architects, developers, DevOps engineers

STATUS

Issued for review and approval

PREPARED BY

Platform engineering team

COMPANION DOCUMENT

Development Completion Document (technical handover)

Contents

1 Executive summary

2 Business context and objectives

3 Scope and user roles

4 Proposed solution architecture

5 Microservices

6 Communication patterns

7 Data architecture

8 Technology stack

9 Infrastructure

10 Security architecture

11 Resilience and non-functional requirements

12 Observability

13 Quality assurance and testing strategy

14 Deployment strategy

15 Delivery plan

16 Risks and mitigations

17 Assumptions and next steps

How to read this document

Sections 1 to 3 describe the business case and are written for every reader. Sections 4 to 9 describe the proposed technical solution. Sections 10 to 14 cover security, quality and operations, and are of most interest to architects and DevOps engineers. Sections 15 to 17 cover the delivery plan, risks and next steps.

1 Executive summary

We propose to build the Food Delivery Platform as a set of small, independently deployable services that communicate through a central API gateway and an event backbone. Customers order food from local restaurants, restaurants manage their menus and orders, delivery partners carry orders to the customer's door, and platform administrators supervise the whole marketplace.

A microservices architecture is proposed because the platform has several clearly separate business domains (users, restaurants, menus, orders, payments, deliveries, notifications, search) that change at different speeds, carry different loads and have different reliability needs. Splitting them lets each domain be built, tested, scaled and released on its own, and keeps a failure in one area (for example the search index) from stopping the others (for example checkout).

16	4	12	18
Spring Boot applications: gateway, 13 domain services, 2 AI services	user roles, each with its own web area	databases: one per service that stores data	delivery phases, each one reviewed before the next starts

Key features of the proposal

- **One entry point.** All traffic enters through an API gateway that routes requests, limits abusive traffic and protects services that are temporarily unavailable.
- **Event-driven integration.** Services publish business events (an order was created, a payment succeeded, a delivery was picked up) to Apache Kafka. Other services react to those events instead of calling each other, which keeps them loosely coupled.
- **Reliable by design.** A transactional outbox guarantees that a change and its event are saved together; a choreographed saga coordinates checkout, payment, refunds and delivery without a central coordinator.
- **Fast discovery.** Restaurant and dish search runs on Elasticsearch, kept up to date from events, with typo-tolerant matching and filters.
- **Built for operations.** Metrics, logs and distributed traces are part of every service from the start, with ready-made dashboards.
- **Quality gates.** Unit, integration and end-to-end tests, architecture rules and static analysis run on every build.
- **AI assistance.** A role-aware conversational assistant (Spring AI with a locally hosted language model) answers questions about orders, menus and deliveries and can prepare actions that a person confirms.

2 Business context and objectives

2.1 Context

Online food delivery connects three groups of people who all depend on the same order: the customer who pays, the restaurant that cooks, and the rider who delivers. Each step has to be visible to the others in near real time, money must never be taken twice or lost, and the platform operator needs to supervise restaurants, riders, refunds and disputes. Demand is also uneven: lunch and dinner peaks can multiply traffic for browsing, checkout and tracking within minutes.

2.2 Objectives

Objective	How the proposal meets it
Independent evolution of business areas	One service per domain with its own database and release cycle; contracts between services are versioned events and REST APIs.
Reliable money handling	Idempotent payment and refund APIs, verified payment-provider callbacks, automatic refunds for cancelled orders, and an immutable audit trail.
Real-time experience	Event-driven status changes pushed to browsers over WebSocket; live delivery tracking on a map.
Scalability at peak times	Stateless services that scale horizontally; Kafka partitions spread event processing; Redis caches hot reads.
Graceful degradation	Timeouts, retries, circuit breakers and fallbacks so that one slow or failed service does not take the platform down.
Operational visibility	Metrics, centralised logs and distributed tracing for every request, from the gateway through every service and event.
Maintainability	Consistent project structure, automated quality gates and documentation per service.

2.3 Success criteria

- A customer can find a restaurant, order, pay, track and receive an order end to end through the web application.
- Restaurants, riders and administrators each have a working area for their daily tasks.
- Every business flow is covered by automated end-to-end tests that run against the deployed platform.
- The loss or slowness of one non-critical service is visible on dashboards and does not stop checkout.

3 Scope and user roles

3.1 User roles

Role	Main capabilities
Customer	Register and sign in; browse and search restaurants and dishes; manage a cart and delivery addresses; apply coupons; check out and pay online or cash on delivery; follow the order and its delivery live; cancel before the restaurant accepts; review delivered orders; receive notifications; chat with support.
Restaurant owner	Register as a restaurant partner and submit a restaurant for approval; manage profile, opening hours and menu (prices, availability); accept or reject incoming orders and move them through preparation; see sales, popular dishes and reviews.
Delivery partner	Register with vehicle and city; go online and offline; receive delivery offers and accept or decline them; navigate to pickup and drop-off; confirm pickup and delivery (including cash collection); see delivery history.
Platform administrator	Dashboard and reports; approve, suspend and manage restaurants, menu items, categories, users and delivery partners; manage coupons; review payments and issue refunds; reassign deliveries; read the platform-wide audit log; answer support chats.

3.2 In scope

- Back-end microservices, API gateway, event backbone and supporting infrastructure.
- A responsive web application (desktop, tablet, phone) with customer, owner, partner and admin areas, in light and dark themes.
- Online payments through a pluggable provider (a simulated provider for development and testing; Razorpay for real payments).
- E-mail, SMS (simulated in development), push and in-app notifications.
- Search, coupons, reviews, support chat and an AI assistant.
- Live delivery tracking: the rider's position from the partner's browser while on the way, on the customer's tracking map.
- Observability, automated testing, code-quality gates, containerisation and local deployment tooling.
- Token-based authentication (JWT) as the final delivery phase.

3.3 Out of scope for this phase

- Native iOS and Android applications (the web application is mobile-friendly and can later be wrapped or complemented).
- Background GPS from a native rider app, and route lines or arrival-time estimates on the map (they need a routing service).
- Rider payouts, restaurant settlements and tax invoicing.
- Production cloud provisioning; the target deployment approach is proposed in section 14.

4 Proposed solution architecture

The platform is organised in four layers: a single web application; an API gateway; independent domain services; and shared infrastructure (event backbone, databases, cache, search, observability). The figure below shows the proposed components and how they connect.

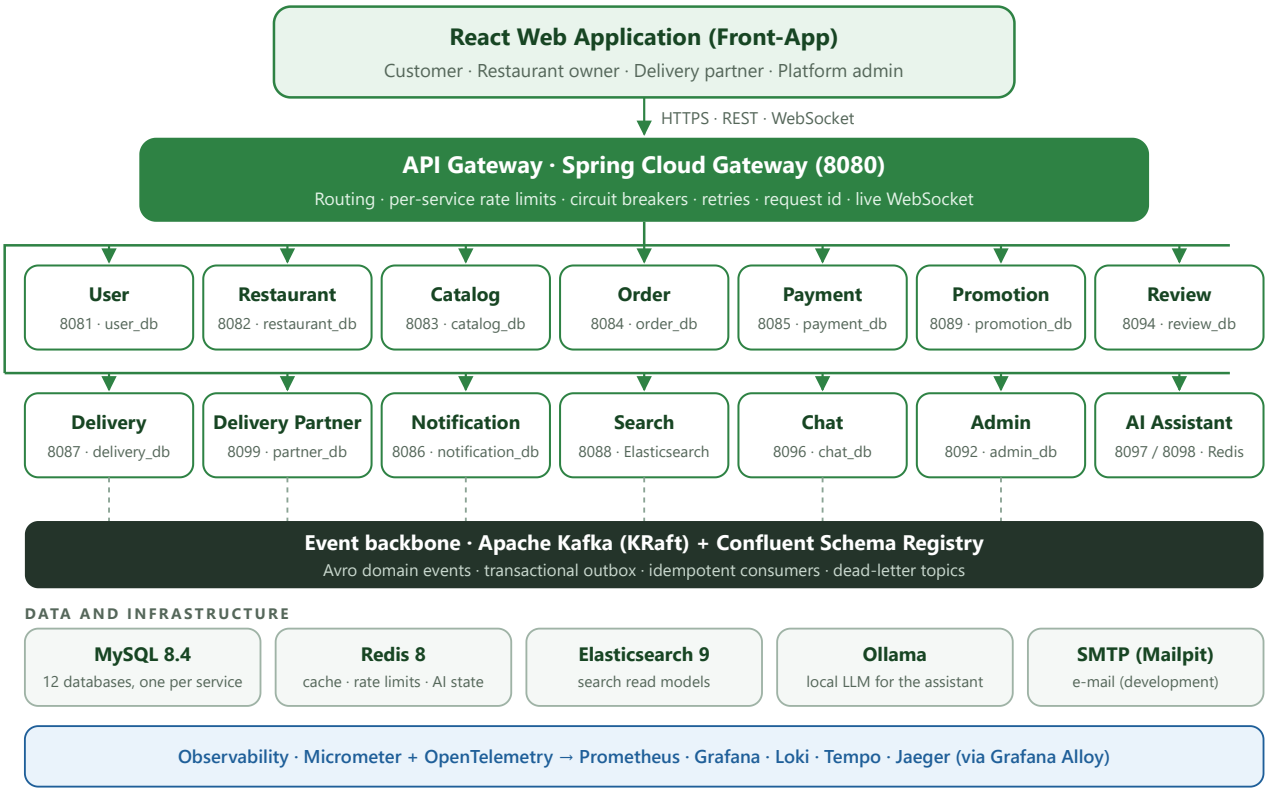


Figure 1. Proposed architecture: clients reach every service through the API gateway; services exchange business events through Kafka; each service owns its data.

4.1 Architecture principles

Principle	What it means in practice
Database per service	Each service owns its tables; no service reads another service's database. Data needed elsewhere is shared through APIs or events.
Single entry point	Browsers talk only to the API gateway, which hides the internal topology and applies cross-cutting protection.
Events for cross-domain change	When something important happens, the owning service publishes an event; interested services react. Services do not instruct each other.
Synchronous calls only when needed	A direct REST call is used only when a fresh answer is required to complete a request (for example, price check at checkout) and is always guarded by timeouts and circuit breakers.
At-least-once delivery, applied once	Events may be delivered more than once; consumers detect and skip repeats so the effect happens once.
Stateless services	No session state in memory; any instance can serve any request, so services scale horizontally.

Principle	What it means in practice
Observable by default	Every request carries a request id and a trace context through REST calls and events.
Contracts first	REST APIs are versioned (`/api/v1`) and documented with OpenAPI; events are Avro schemas checked for compatibility by a schema registry.

4.2 Why microservices

A single large application would be simpler to start but would force every change through one release, one database and one scaling unit. The proposed split follows business capabilities, so each team or developer can work on one domain with a small, understandable code base. The added complexity of distribution (network failures, eventual consistency, operations) is addressed explicitly by the communication patterns in section 6, the resilience measures in section 11 and the observability in section 12.

5 Microservices

Thirteen domain services, the API gateway and two AI services make up the platform. Each is a separate Spring Boot application with its own build, tests, container image and (where it stores data) its own MySQL database.

Service	Responsibility	Data	Port
API Gateway	Single entry point: routing, per-service rate limiting, timeouts, retries, circuit breakers, request id, WebSocket for live updates.	Redis (rate-limit counters)	8080
User	Registration, sign-in, profiles, addresses, roles and account status; bootstraps the first administrator.	user_db	8081
Restaurant	Restaurant onboarding and lifecycle (pending, approved, active, suspended), profile and opening hours.	restaurant_db	8082
Catalog	Global food categories and restaurant menus: dishes, prices, availability, admin blocking.	catalog_db	8083
Order	Cart, checkout, order lifecycle and timeline, restaurant order handling, cancellations; owner of the checkout saga's order side.	order_db	8084
Payment	Payments and refunds through a pluggable provider (simulated or Razorpay), webhooks, automatic refunds.	payment_db	8085
Notification	E-mail, SMS, push and in-app inbox driven by events; live updates to browsers over WebSocket.	notification_db	8086
Delivery	Deliveries and dispatch: assigning orders to available partners, offers, pickup, delivery, partner work status.	delivery_db	8087
Delivery Partner	Delivery partners' registration, profile (vehicle, city) and account suspension.	delivery_partner_db	8099
Search	Restaurant, dish and category search with typo tolerance, facets and filters.	Elasticsearch indexes	8088
Promotion	Coupons: rules, validity, usage limits, reservation at checkout.	promotion_db	8089
Review	Customer ratings and reviews of delivered orders and their dishes.	review_db	8094
Chat	Support conversations between users and the platform's support team.	chat_db	8096
Admin	Administrator dashboard and reports assembled from the owning services; platform-wide audit log.	admin_db	8092
AI client (BFF)	The assistant's browser-facing side: verifies the caller, keeps conversations, applies limits.	Redis	8097
AI server	Spring AI agent and tools; calls business APIs only through the gateway, as the user.	Redis; Ollama model	8098

5.1 Shared libraries

One shared library, `event-contracts`, holds the Avro schemas of every business event and the topic names. It is versioned and installed like any other dependency; it contains no business logic, so services remain independent.

5.2 Why these boundaries

- **Order and payment are separate** because payments have their own provider integration, security requirements and reconciliation, and must never be charged twice by a retried order request.
- **Delivery and delivery partner are separate**: who a partner is (profile, suspension) changes rarely, while dispatch (who is free, who gets which order) must happen inside one database transaction to guarantee that no partner receives two deliveries.
- **Search is a read model**: it owns no business data; it is rebuilt from events and can be replaced or re-indexed without touching the source services.
- **Admin owns no business data**: it composes figures from the services that own them and keeps a copy of every audit record for one platform-wide audit view.

6 Communication patterns

6.1 Client to platform: REST through the gateway

- Versioned JSON APIs under `/api/v1`; administrator APIs under `/api/v1/admin`.
- Errors follow RFC 9457 (`ProblemDetail`), with business messages that are safe to show to users.
- Lists are paginated on the server (page, size, sort; at most 100 per page).
- Every response carries an `X-Request-Id` that follows the request through all services and events.
- State-changing operations that must not be repeated (placing an order, paying, refunding) take an `Idempotency-Key`: a retried request returns the original result instead of acting twice.

6.2 Service to service: synchronous REST, guarded

A service calls another directly only when it needs a fresh answer to complete the current request: for example, checkout confirms prices with the catalog and reserves a coupon with the promotion service. Every such call has a timeout, a retry only when the connection itself failed, a circuit breaker per target service and a bulkhead limiting concurrent calls (details in section 11).

6.3 Asynchronous events: Kafka with Avro

When a service changes something other services care about, it publishes an event. Events are defined as Avro schemas and registered in Confluent Schema Registry, which rejects schema changes that would break existing consumers. Each topic has three partitions; events are keyed by entity (for example the order id), so all events of one order are processed in order. A consumer that cannot process an event retries it, then moves it to a dead-letter topic so the stream is never blocked.

Event (examples)	Published by	Consumed by
UserRegistered	User	Notification, Delivery Partner
RestaurantChanged	Restaurant	Catalog, Notification, Delivery, Search
MenuItemChanged, CategoryChanged	Catalog	Search
OrderCreated, OrderStatusChanged, OrderCancelled	Order	Payment, Delivery, Notification, Promotion
PaymentStatusChanged	Payment	Order, Notification
DeliveryStatusChanged	Delivery	Order, Notification
DeliveryPartnerChanged / DeliveryPartnerStatusChanged	Delivery Partner / Delivery	Delivery / Delivery Partner
AdminActionAudited	Every service with admin actions	Admin
ChatMessageSent, ChatMessagesRead	Chat	Notification (live updates)

6.4 Transactional outbox and idempotent consumers

Writing to the database and publishing to Kafka are two separate systems; doing both in a request risks saving a change without its event, or the other way round. The outbox pattern removes that risk: the service writes the event into its own `outbox_events` table *in the same database transaction* as the change, and a relay publishes it to Kafka shortly afterwards. If Kafka is unavailable, requests still succeed and events wait. Consumers record each processed event id, so an event delivered twice is applied once.

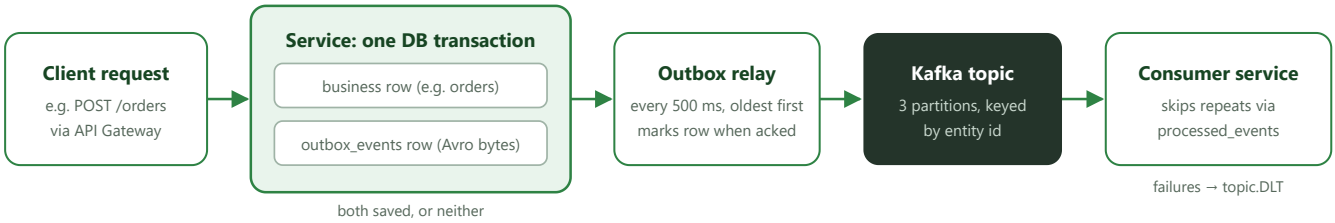


Figure 2. Transactional outbox: a change and its event are saved together; the relay publishes later.

6.5 Saga choreography for checkout

An order spans several services (order, payment, restaurant, delivery, promotion, notification). Instead of a central coordinator, each service reacts to the previous step's event: order created, payment succeeded, order placed, food ready, delivery assigned, picked up, delivered. Failures trigger compensating steps: a cancelled paid order is refunded automatically, a coupon use is given back, a delivery is cancelled and its partner freed, and an unpaid order is cancelled after a time limit.

6.6 Real-time updates

Browsers keep one WebSocket connection through the gateway. The notification service turns business events into small live messages (order moved to preparing, delivery picked up, new support message), so pages update without polling. Pages fall back to periodic refresh when the connection is lost.

6.7 Read models for search

The search service builds its Elasticsearch indexes from restaurant and catalog events (a CQRS-style read model). MySQL remains the source of truth; a consistency check can detect and repair any drift, and indexes can be rebuilt by replaying the current state.

7 Data architecture

Store	Use	Consistency and recovery
MySQL 8.4	System of record. One database and one database user per service; schemas managed by versioned Flyway migrations shipped with each service.	ACID transactions inside a service; backups per database.
Kafka	Durable event log between services; dead-letter topics for events that cannot be processed.	Events kept for replay; consumers idempotent.
Elasticsearch	Search indexes for restaurants, dishes and categories.	Rebuilt from events; consistency check and repair.
Redis	Cache of hot reads (menus, categories, restaurant pages), rate-limit counters, AI conversations.	Disposable: the platform works without it; data can be lost and rebuilt.

7.1 Local copies instead of chatty calls

Where a service needs another's data frequently, it keeps a small local copy updated from events: the catalog keeps a snapshot of each restaurant, payment keeps the orders awaiting payment, delivery keeps pickup and drop-off addresses. This keeps menus working while the restaurant service is down, payments possible while the order service restarts, and started deliveries completable during short outages.

7.2 Consistency model

Inside a service, data is strongly consistent. Across services the platform is eventually consistent: for example, an order moves to "placed" a moment (usually well under a second) after the payment succeeds. Concurrent conflicting changes are detected by optimistic locking and rejected with a clear "please retry" answer.

7.3 Auditing

Every administrator action (approve a restaurant, block a dish, refund a payment, reassign a delivery) is written to an immutable audit table in the same transaction as the change, with who, when, why and the request id. A copy is sent to the admin service for a single platform-wide audit view.

8 Technology stack

Area	Technology	Reason
Language and runtime	Java 21 (LTS)	Long-term support; records, virtual threads
Service framework	Spring Boot 4.1, Spring Cloud 2025.1	Industry standard; production features built in
API gateway	Spring Cloud Gateway 5 (reactive)	Routing, filters, rate limiting, WebSocket
Persistence	MySQL 8.4, Spring Data JPA, Flyway	Reliable relational store; versioned migrations
Messaging	Apache Kafka 4.2 (KRaft), Confluent Schema Registry 8.2, Avro 1.12	Durable, ordered event log; safe schema evolution
Search	Elasticsearch 9.4	Full-text and faceted search
Cache and rate limits	Redis 8	Fast, simple, disposable
Resilience	Resilience4j 2.4	Timeouts, retries, circuit breakers, bulkheads, rate limiters
Observability	Micrometer, OpenTelemetry, Prometheus, Grafana, Loki, Tempo, Jaeger, Grafana Alloy	Metrics, logs and traces linked together
AI	Spring AI 2, Model Context Protocol, Ollama (local model)	Assistant without sending data to third-party AI services
Payments	Pluggable provider: simulated gateway, Razorpay Checkout	Test without real money; real payments via Razorpay
Web application	React 19, TypeScript, Vite, Redux Toolkit, TanStack Query, Tailwind CSS, Leaflet with OpenStreetMap	Modern, typed, fast; no paid map API
Quality	JUnit 5, Mockito, Testcontainers, ArchUnit, Checkstyle, SpotBugs, JaCoCo, SonarQube, Vitest	Automated gates on every build
Build and packaging	Maven (wrapper per service), Docker multi-stage images, Docker Compose	Reproducible builds; one image per service

9 Infrastructure

Component	Role	Development set-up
MySQL 8.4	Relational databases, one per service (created with dedicated users)	One container, initialised by script
Kafka (KRaft) + Schema Registry	Event backbone and schema governance	Single broker; topics created by the services
Redis	Cache, rate limits, AI conversation state	No persistence; bounded memory
Elasticsearch	Search indexes	Single node
Ollama	Local large language model for the assistant (CPU or GPU)	Bound to the local machine only
Mail server	Outgoing e-mail	Mailpit catches every e-mail for inspection
Observability stack	Prometheus, Grafana, Loki, Tempo, Jaeger, Grafana Alloy collector	Optional containers with provisioned dashboards
Data UIs	Kafka UI, Kibana, phpMyAdmin	Optional, development only
Code quality	SonarQube with PostgreSQL	Optional profile, on demand
Change data capture (optional)	Debezium on Kafka Connect, in shadow mode for evaluation	Optional profile

Resource planning

Running every service, the full observability stack and the language model on a single developer workstation needs more than 16 GB of memory for containers. The proposed development set-up therefore runs the infrastructure in Docker and the services as local processes, with the observability stack started only when needed. Production environments size each component separately (section 14).

10 Security architecture

10.1 Identity and access

Security is delivered in two steps so that functional work is not blocked:

1. **Development phases (interim).** Users sign in with e-mail and password, verified by the user service (passwords stored as BCrypt hashes). The web application then sends the signed-in user's id and role in request headers. This is suitable only for development and testing environments that are not exposed to the internet.
2. **Final phase: token-based security (proposed).** Spring Security with JSON Web Tokens: the user service (or an identity provider) issues signed access tokens; the API gateway validates every token and passes the verified identity to the services; services reject requests that did not come through the gateway. Refresh tokens, token expiry and sign-out are included. Because identity is resolved in one place per service, this change does not alter business code.

10.2 Authorisation

Role-based access control with four roles: `CUSTOMER`, `RESTAURANT_ADMIN`, `DELIVERY_PARTNER` and `PLATFORM_ADMIN`. Every service checks the role for each endpoint and enforces ownership in its own business logic (an owner sees only their restaurant's orders, a partner only their own deliveries, a customer only their own orders). Nobody can register as an administrator; the first administrator is created from protected configuration.

10.3 Protection of data and transactions

- **Transport:** HTTPS (TLS) at the edge in every non-local environment; internal traffic on a private network.
- **Payments:** card details never touch the platform; the provider's hosted checkout handles them. Payment callbacks are accepted only after their signature is verified (HMAC) and the provider confirms the captured amount.
- **Idempotency:** orders, payments and refunds accept an idempotency key so that a retried request cannot charge or refund twice.
- **Secrets:** passwords and keys come from environment configuration (a secrets manager in production), never from source code.
- **Input validation:** every request body is validated; errors never expose stack traces or internal details.
- **Abuse protection:** per-client rate limits at the gateway, stricter limits for sign-in and registration.
- **Auditing:** immutable audit records of every administrator action, with the reason given.

10.4 AI assistant safeguards

- The assistant's identity comes from the verified session, never from what the user types or what the model produces.
- Each role is offered only its own tools; the business services check permissions again on every call.
- The model cannot change anything by itself: it can only prepare an action (place or cancel an order, accept an order, mark a delivery) that the user must confirm with a button.
- The language model runs locally, so conversation data is not sent to third-party AI services.

11 Resilience and non-functional requirements

11.1 Resilience measures

Where	Pattern	Behaviour
Service-to-service calls	Timeout	2 s to connect, 3 to 4 s to answer: no request waits forever.
	Retry	One more attempt after 200 ms, only when the connection failed and the call is safe to repeat.
	Circuit breaker	Opens after repeated failures or slow calls; callers get a fast "unavailable" answer while the target recovers.
	Bulkhead	At most 20 concurrent calls to one service, so a slow dependency cannot exhaust all threads.
Payment provider	Rate limiter and circuit breaker, no retry	Stays within the provider's limits; a charge is never repeated automatically.
API gateway	Rate limit, timeouts, retry (reads only), circuit breaker, fallback	Per-service settings; a clear 503 or 504 with `Retry-After` instead of hanging.
Event processing	Retries and dead-letter topics	A poisonous event never blocks the stream; it is set aside for analysis.
Fallbacks	Degraded answers	For example: the cart still loads when coupons cannot be checked; the admin dashboard shows the figures that are available.

11.2 Non-functional requirements

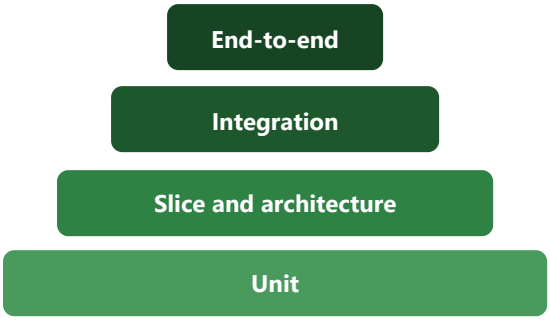
Quality	Target and approach
Scalability	Stateless services scale horizontally behind the gateway; Kafka partitions allow parallel consumers; Redis caching for hot reads.
Availability	No single domain failure stops checkout (except the order and payment services themselves); multiple instances per service in production.
Performance	Pagination everywhere; caching of menus and categories; search offloaded to Elasticsearch; response-time percentiles monitored per endpoint.
Reliability	Outbox, idempotent consumers, idempotency keys and compensating actions guarantee correct outcomes despite retries and outages.
Maintainability	Uniform service structure, architecture rules enforced by tests, documentation and API files per service.
Usability and accessibility	Responsive design, light and dark themes, keyboard support and readable contrast in the web application.

12 Observability

Every service exposes metrics, writes structured logs and takes part in distributed tracing from day one.

Signal	Collected with	What it answers
Metrics	Micrometer → Prometheus	Traffic, errors and latency per endpoint; JVM and connection pools; Kafka lag; outbox backlog; circuit breakers; business counters (orders, payment failures, delivery times).
Logs	Container logs → Grafana Alloy → Loki	Every line carries the trace id, request id and the user, order, payment or delivery id where known.
Traces	OpenTelemetry → Alloy → Tempo and Jaeger	One trace per business transaction across the gateway, REST calls and Kafka events, including the outbox.
Dashboards	Grafana (provisioned)	"Services" (technical health) and "Business" (orders, outcomes, refunds, delivery times); links from a metric to its trace and logs.

13 Quality assurance and testing strategy



Business flows across the running platform, through the gateway.

Per service, with real MySQL, Kafka, Redis and Elasticsearch (Testcontainers).

Web layer: access rules, validation, error format; architecture rules.

Business rules: state transitions, prices, limits, retries.

Gate	What it enforces
ArchUnit	Layering (controller → service → repository), no entities in APIs, constructor injection, no forbidden constructs.
Checkstyle	Code style and naming conventions shared by every service.
SpotBugs	Bug patterns in compiled code.
JaCoCo	A per-service minimum test coverage that may not drop.
SonarQube	A shared quality gate with history.
Front-end checks	Type checking, linting, formatting and component tests on every build.

Each end-to-end test creates its own data (owner, restaurant, customers, partner) in its own city through the public APIs, so tests are independent of existing data and of each other, and can run against any environment.

14 Deployment strategy

14.1 Environments

Environment	Purpose	Characteristics
Development	Daily development and debugging	Infrastructure in Docker, services run locally; simulated payments; mail catcher.
Demo	Showcasing to stakeholders	Stable data set; simulated payments.
UAT	Client acceptance testing	Production-like configuration; payment provider test mode; full end-to-end suite.
Pre-production	Final rehearsal of a release	Same topology and sizing as production; token-based security enabled.
Production	Live service	High availability, managed data services, monitoring and alerting.

The web application is built per environment from its own configuration file (gateway address, feature flags), so the same code moves through every stage.

14.2 Packaging

- Every service has a multi-stage Dockerfile: the build stage compiles and packages; the runtime stage uses a slim Java runtime, a layered JAR for fast rebuilds, and runs as a non-root user.
- Docker Compose describes the infrastructure and, through profiles, optional groups (services, AI, change data capture, code quality). A script starts the services locally for development.
- Database schemas migrate automatically at service start-up (Flyway), forward-only and backward compatible.

14.3 Proposed production topology

Recommendation (to be confirmed with the client's hosting choice)

- **Orchestration:** a managed Kubernetes service (for example AKS, EKS or GKE) with one deployment per service, at least two replicas each, readiness and liveness probes on the existing health endpoints, and horizontal autoscaling on CPU and request rate.
- **Edge:** an ingress controller or cloud load balancer terminating TLS in front of the API gateway; a CDN for the web application's static files.
- **Data services:** managed MySQL with automated backups and point-in-time recovery; managed Kafka with Schema Registry; managed Redis; managed Elasticsearch (or OpenSearch).
- **AI model:** a GPU node pool or a dedicated model host for responsive assistant answers.
- **Secrets and configuration:** a secrets manager (for example Azure Key Vault, AWS Secrets Manager or HashiCorp Vault) injected as environment variables.
- **Observability:** the same Prometheus, Grafana, Loki and Tempo stack (self-hosted or a managed Grafana offering) with alert rules for errors, latency, consumer lag and outbox backlog.

14.4 Proposed CI/CD pipeline

1. **Build and test** each changed service: unit, slice and integration tests, architecture rules, Checkstyle, SpotBugs, coverage.
2. **Quality gate** in SonarQube; the pipeline stops on failure.
3. **Container image** built once, tagged with the commit, scanned for vulnerabilities and pushed to a registry.
4. **Deploy to UAT** automatically; run the end-to-end suite against it.
5. **Promote** the same image to pre-production and production after approval, with rolling or blue-green deployment and automatic rollback when health checks fail.

15 Delivery plan

The platform is delivered in phases. Each phase ends with a working, tested increment and a review before the next phase starts.

Phase	Deliverable	Content
1	Foundation and API gateway	Repository layout, conventions, gateway, infrastructure containers.
2	User service	Registration, sign-in, profiles, addresses, roles, administrator bootstrap.
3	Restaurant service	Onboarding workflow, approval, lifecycle, opening hours.
4	Catalog service	Categories, menus, prices, availability.
5	Order service	Cart, checkout, order lifecycle, restaurant order handling.
6	Payment service	Payments, refunds, provider integration.
7	Kafka and Schema Registry	Event contracts, producers and consumers.
8	Saga and outbox	Transactional outbox, idempotent consumers, checkout saga and compensations.
9	Notification service	E-mail, SMS, push, in-app inbox, live updates.
10	Delivery service	Dispatch, offers, pickup and delivery, partner status.
11	Search service	Elasticsearch indexes, search APIs, consistency repair.
12	Redis	Caching and rate limiting.
13	Promotions and administration	Coupons; admin dashboard, reports, platform-wide audit log.
14	Resilience and observability	Resilience4j patterns; metrics, logs, traces, dashboards.
15	Quality and integration testing	Quality gates; end-to-end test suite.
16	Web application: admin area	Dashboard, orders, customers, foods, restaurants, coupons, payments, deliveries, reports, audit.
17	Web application: customer area	Browse, search, cart, checkout, payment, orders, tracking, profile.
18	Security	Token-based authentication and authorisation (JWT) across gateway and services.

Extensions planned alongside the phases: restaurant-owner and delivery-partner web areas, a separate delivery-partner service, reviews and ratings, support chat, real payment provider (Razorpay), live map tracking and the AI assistant.

16 Risks and mitigations

Risk	Impact	Mitigation
Complexity of a distributed system	High	Uniform service structure; distributed tracing and request ids; end-to-end tests of every business flow; clear documentation.
Inconsistent data between services	High	Transactional outbox, idempotent consumers, compensating actions, consistency checks for read models.
Payment errors (double charge, lost payment)	High	Idempotency keys, no automatic retry of charges, verified provider callbacks, automatic refunds, audit trail.
Interim identity headers exposed publicly	High	Interim mode only in closed environments; token-based security (phase 18) before any public deployment.
Event schema changes breaking consumers	Medium	Schema Registry compatibility checks; versioned shared contract library.
Infrastructure cost and operational load	Medium	Managed data services; autoscaling; optional components (observability UIs, change data capture) enabled only where needed.
AI answers being wrong or misleading	Medium	Answers grounded in tool results only; figures checked against tool data; consequential actions always confirmed by a person.
Developer workstation resource limits	Low	Services run locally, infrastructure in Docker, heavy tools started on demand.

17 Assumptions and next steps

17.1 Assumptions

- The client provides payment provider accounts (test and live keys) and a mail and SMS provider for production.
- The hosting platform (cloud provider or on-premises) is decided before the pre-production environment is built.
- The web application is the primary client; native mobile apps can reuse the same APIs later.
- Real-time rider GPS requires a rider mobile application or device integration, delivered as a follow-up.

17.2 Next steps

1. Review and approve this proposal, including the security approach and the delivery plan.
2. Confirm the hosting platform and managed services for UAT, pre-production and production.
3. Provide payment, mail and SMS provider credentials for the test environments.
4. Agree the acceptance criteria for each phase and the UAT schedule.

> **Companion document**
>
> The **Development Completion Document** describes the platform as implemented: services, project structure, interactions, workflows, configuration, testing, deployment and the completed feature set.